

ADVANCED & SPECIALIZED QUESTIONS

Security Hardware Design Verification Engineer Interview

VERIFICATION METHODOLOGY & UVM MASTERY

Q: What is UVM and why critical for ASIC verification?

A: UVM is IEEE standard SystemVerilog class library providing modular, reusable verification infrastructure. Importance: (1) Scalability—handles designs from simple blocks to billion-transistor SoCs, (2) Modularity—driver/monitor/sequencer/agent components pluggable, (3) Reusability—base classes for rapid development, (4) Standardization—industry consensus, team mobility, (5) Automation—built-in reporting, coverage, randomization. Modern SoC functional verification critical path; UVM enables 95%+ coverage required for production.

Q: UVM object hierarchy and roles explained

A: Hierarchy: `uvm_object` (base, printing/copying), `uvm_component` (phase management, reporting), `uvm_driver` (pin-level stimulus from transactions), `uvm_monitor` (observes pins, creates transactions), `uvm_sequencer` (generates sequences, constraint randomization), `uvm_agent` (bundles driver/monitor/sequencer), `uvm_scoreboard` (compares expected vs actual), `uvm_environment` (integrates agents/scoreboards), `uvm_test` (top-level, configures environment). Clear separation of concerns enables modular verification.

Q: Constrained-random vs directed testing tradeoffs

A: Constrained-random: generates stimulus via RNG within constraints. Advantages—broad input-space exploration, finds unexpected corners, probabilistically hits interesting scenarios. Disadvantages—may miss specific cases, many iterations needed. Best for checking design doesn't break arbitrarily. Directed: manually-written test cases targeting specific functionality, corners, errors. Advantages—ensures critical paths, targets known-bug scenarios, efficient. Disadvantages—manual effort, requires engineer knowledge. Best for features, error handling. Optimal: hybrid—constrained-random baseline + directed for known corners and critical features.

Q: Functional vs code coverage explained

A: Code coverage (structural): which RTL lines/branches execute. Advantages—ground truth, automated by tools. Limitations—code execution $\neq$ correctness; wrong code can be covered. Functional coverage (semantic): which behaviors/features exercised. Advantages—better understanding of completeness, engineer decides what matters. Limitations—manual definition, no tool automation. Ideal: both 95%+. High code coverage, low functional coverage suggests implementation tested but features not. Low code coverage indicates untested functionality.

Q: Gate-level simulation (GLS) importance

A: GLS vs RTL: RTL is behavioral, gate-level is post-synthesis with actual gates and timing. Why GLS matters: (1) Timing issues don't appear in RTL (idealized delays), (2) Glitches from hazards, (3) Metastability, CDC issues, (4) Accurate power estimation, (5) Sign-off confidence—proves post-synthesis works. Process: synthesize with standard library, extract SDF (delays), run identical tests RTL and GLS, compare waveforms. GLS slow but essential for production ASICs.

Q: Debugging failing tests in complex UVM testbench

A: Methodology: (1) Reproduce consistently—identify failing test, run in isolation, note random seed, (2) Scope failure—simulator error or functional check fail? (3) Instrument—uvvm_info statements, transaction logging, waveform dumps, (4) Waveform analysis—identify signal divergence, correlate with logs, (5) Hypothesize and test—form hypothesis, add targeted checks, (6) Iterate. Common issues: timing (clocking, synchronization), protocol violations, reference model bugs, over-constraining.

Q: SystemVerilog assertions (SVA) role

A: Assertions are boolean expressions checked automatically. Types: (1) Immediate—checked at evaluation time (simulation), (2) Concurrent—properties spanning multiple cycles for protocol checking. Benefits: (1) Automated checking—immediate violation detection, (2) Documentation—encode design intent, (3) Formal verification—properties formally proven. Usage: verify protocol compliance, state consistency, functional properties, timing. Challenges: writing good properties requires design understanding; over-assertion slows simulation.

Q: Reference models in verification

A: Simpler, correct implementation used as ground truth. Where: typically in scoreboard—compares DUT vs reference. Languages: C/C++, Python (external), SystemVerilog (integrated). Requirements: correctness, completeness, bit-exact (crypto/DSP), approximate (others). Building: implement specification directly (AES/DSP), or use senior engineer understanding (custom logic), use published standards (NIST, IEEE). Limitations: reference model can have bugs—must be carefully reviewed. Example: AES reference model implements FIPS 197, scoreboard compares outputs.

Q: Clock domain crossing (CDC) verification

A: Problem: data moving between asynchronous clocks risks metastability (flip-flop samples when setup/hold violated). Solution: synchronizer chain (2-3 flip-flops)—statistically reduces metastability to acceptable levels. CDC verification: (1) Structural checks—verify synchronizers on all cross-domain signals, (2) Metastability simulation—inject random delays, verify recovery, (3) Functional—synchronized signals behave correctly, (4) Timing—synchronizer delay accounted for. Tools: Verplex, Xcelium CDC. CDC analysis required for all multi-clock designs.

SECURITY HARDWARE VERIFICATION IN DEPTH

Q: Cryptographic hardware accelerators verification

A: Purpose: accelerate crypto operations (AES 100x faster than software). Verification: (1) Functional—known-vector testing (NIST vectors), reference model, property-based

(invertibility), edge cases, (2) Performance—throughput/latency targets, (3) Power—power budget, dynamic/leakage analysis, (4) Side-channel—power/timing independent of secrets, formal proofs, (5) Fault injection—detect/mask injected faults, (6) Integration—interface to CPU, interrupts, DMA, multi-operation sequences. Multi-dimensional verification essential.

Q: Side-channel analysis and resistance

A: Attacks: (1) Power analysis—power consumption correlates with secrets, differential power analysis (DPA) uses statistical correlation to recover keys, (2) Timing—execution time depends on secrets (branch prediction, cache), (3) EM—electromagnetic emissions leak information, (4) Fault injection—injecting bit flips help deduce computations. Verification: (1) Power traces—simulate with known plaintext, capture toggling activity, check correlation (should be ~zero), (2) Timing—constant cycle count for all inputs, (3) Formal—prove output independent of secrets, (4) EM simulation—advanced, (5) Fault injection—bit flips detected or masked. Challenge: demonstrating resistance requires sophisticated analysis.

Q: Trusted Execution Environment (TEE) verification

A: TEE components: secure processor, secure memory, crypto accelerators, RNG, secure clock. Verification: (1) Functional—mode switching (normal↔secure), context save/restore, SMC interface, secure memory access enforcement, peripheral access control, (2) Security— isolation (secure isolated from normal), confidentiality/integrity, authentication, (3) Side-channel—mode switch timing constant, power independent, covert channels through shared resources (cache, memory bus), (4) Formal—formally prove isolation properties, (5) Certification—map to Common Criteria/EAL5+ requirements. Complex, multi-dimensional verification.

Q: Hardware random number generators (RNGs)

A: Types: (1) DRNG (deterministic)—pseudo-random from seed, passes statistical tests, (2) TRNG (true)—physical entropy source (oscillator jitter, thermal noise), non-deterministic, (3) Hybrid—TRNG + DRNG post-processing. Verification: (1) Entropy rate—should contain $\log(2)$ bits entropy per bit, (2) Statistical tests—NIST SP 800-22 suite (frequency, block frequency, runs, DFT, linear complexity, etc.), (3) Bias detection—equal 0s/1s, (4) Correlation—output bits uncorrelated, (5) Cryptographic strength—post-processing (SHA-256 hashing) produces strong randomness, (6) Failure detection—TRNG halts and flags when entropy source fails. Verification essential for cryptographic quality.

Q: Secure key storage and management

A: Storage mechanisms: (1) OTP fuses (one-time programmable, burned at manufacturing, immutable), (2) Secure SRAM (volatile, keys exist during operation, cleared on reset), (3) Key cache (temporary, actively-used keys), (4) Encrypted storage (keys stored encrypted, decrypted in secure zone). Verification: (1) Storage integrity—keys retrievable without corruption, ECC/parity protection, (2) Access control—only authorized read, non-secure can't access, DMA blocked, (3) Key lifecycle—generation (TRNG), storage, usage (secure context), destruction (overwritten), (4) Side-channel—access patterns constant-time, power/EM independent of keys, (5) Formal—prove no instruction sequence leaks keys, (6) Testing—attempt unauthorized access, fault injection, durability over time. Critical security component.

Q: Secure boot and root of trust (RoT)

A: RoT: trusted hardware-based component holding master keys, verifies code signatures before execution. Secure boot flow: power-on → immutable boot ROM (RoT) from OTP fuses → verify bootloader signature → bootloader verifies kernel → only authenticated code executes. Verification: (1) Signature verification—crypto signatures (RSA, ECDSA) correct, signature database protected, (2) Hash verification—code hash matches expected, (3) Certificate chains—X.509 chains validated, CRL checked, (4) Rollback protection—can't downgrade to vulnerable firmware, version counter in secure storage, (5) Failure handling—boot halts on signature fail (no fallback), error indication (LED), (6) RoT integrity—not modifiable after manufacturing, (7) Formal—RoT implements verification correctly, (8) Functional testing—valid firmware succeeds, invalid/tampered fails, downgraded fails. Sign-off critical for device security.

Q: Physical attack protection verification

A: Attacks: (1) Fault injection—voltage/clock glitches, EM pulses, laser faults, temperature extremes corrupt computation, skip security checks, (2) Side-channels—power, timing, EM leak secrets. Verification: (1) Fault injection—simulate bit flips at random times/locations, verify either detected (ECC, error flag) or output obviously wrong, formal proofs of fault-resistance, (2) Hardening—redundant computation, error detection (parity, ECC), temporal/spatial redundancy, (3) Test scenarios—faults during key operations, comparisons, branching, (4) Temperature/voltage extremes—verify operation across PVT corners, (5) Brute-force—key space sufficiently large (theoretical analysis). Resilience to physical attacks critical for secure devices.

MULTI-CORE & SYSTEM-LEVEL VERIFICATION

Q: Multi-core SoC with security subsystem

A: Complexity: multiple CPUs, GPUs, security cores, peripherals, shared interconnect, caches. Verification: (1) Modular—verify cores in isolation, then integration, (2) Functional—CPU instruction execution, crypto operations, interrupt routing, memory coherency, (3) Performance—no bottlenecks, throughput targets, acceptable latencies, (4) Security—secure/non-secure isolation with multiple cores, cross-core interference prevented, (5) Power—distribution correct, gating per core, clock domain crossings, (6) Formal—isolation properties, coherency protocol, arbitration logic. Challenges: test complexity (constrained-random + directed), simulation speed (may need emulation), multi-core synchronization/memory ordering, debugging multi-core state.

Q: Formal verification for security IP

A: Approaches: (1) Model checking—exhaustive state exploration, prove properties hold in all states, (2) Theorem proving—mathematical proofs, (3) Symbolic execution—explore all paths symbolically. Security properties: (1) Information flow—secrets don't influence public outputs (no leakage), (2) Access control—only authorized access, (3) Isolation—secure world separated from non-secure, (4) Authentication—code signatures always validated, (5) Cryptographic correctness—AES output matches spec. Tools: JasperGold, Verplex, SAT/SMT solvers. Limitations:

state explosion, abstraction required, specification must be correct, scalability. Typical: formal for critical subsystems, simulation for system integration.

Q: Verification completeness and convergence

A: Metrics: (1) Code coverage (95%+), functional coverage (90%+), (2) Coverage growth curve—should saturate (diminishing returns), (3) New bugs found—early tests many, later few (convergence indicator), (4) Bug discovery rate—bugs/test ratio decreases (convergence when near zero), (5) Risk assessment—prioritize high-risk areas, (6) Problem heatmap—extra coverage for bug-prone areas. Sign-off: code >95%, functional >90%, critical properties proven, GLS passes, no high-priority bugs, risk areas covered, team consensus. Post-tape-out analysis informs next project.

ADVANCED PROJECT-SPECIFIC SCENARIOS

Q: SERV bit-serial verification corner cases

A: Challenges: (1) 32-cycle instruction processing—state management across long sequences, (2) Cycle accounting—track latencies, ensure correct, (3) Shift operations—data-dependent cycle counts, difficult to verify end-to-end, (4) Stochastic extension—approximate outputs, statistical validation needed, (5) Pipeline interaction—instructions in-flight at different stages, (6) CSR handling—traps, interrupts, save/restore state. Verification: instruction-level abstraction hiding 32-cycle detail, state machine coverage, reference model, comprehensive corner cases, stochastic-specific coverage.

Q: AES constant-time implementation verification

A: Critical: ensure execution time independent of key/plaintext (prevents timing side-channels). Verification: (1) Code flow analysis—no data-dependent branches, (2) All paths same cycle count—instrument RTL with cycle counters, measure for different inputs, (3) Timing assertions—add SVA checking cycle count constant, (4) Formal verification—prove all input combinations have same latency, (5) Post-silicon measurement—actual hardware latency with power supply variation. AES typically constant-time by design (no early termination, all rounds always execute).

Q: RV32I branch prediction verification

A: If processor has branch prediction: (1) Verify prediction logic correct (branch history table, pattern matching), (2) Prediction accuracy—measure correct vs incorrect (should be 80%+), (3) Misprediction handling—pipeline flush works, execution resumes correctly, (4) Corner cases—back-to-back branches, nested branches, pattern saturation, (5) Functional correctness—despite prediction, final result always correct (prediction is optimization, not correctness). Branch prediction verification adds complexity but optional feature.

Q: TinySoC under memory pressure

A: Scenario: CPU accessing memory while UART/Timer also access. Verification: (1) Bus arbitration—no contention, correct transactions flow, (2) Memory timing—meet timing constraints with concurrent access, (3) Cache effects—if present, coherency maintained, (4)

Interrupt latency—CPU memory ops may delay interrupt response, acceptable? (5) Back-to-back operations—pipelined access works, (6) Error conditions—timeout if memory stuck, error reporting. Tests: high-activity workloads, stress tests, scenario-based.

INTERVIEW POSITIONING & COMMUNICATION STRATEGIES

Q: How to position your projects for Qualcomm role

A: SERV→Security IP bridge: CPU architecture understanding, RTL skills, GLS expertise, comprehensive testbenches directly applicable to verifying cryptographic cores and secure processors. AES→Security IP bridge: UVM testbench structure, reference models, known-vector testing, functional coverage patterns directly scale to other crypto IP. Emphasize: modular thinking, rigor, GLS proficiency, problem-solving, learning velocity. Connect specifics: 'My SERV GLS debugging experience taught me to think systematically about timing—directly applicable to verifying post-synthesis security IP where margins are tight.'

Q: How to handle 'tell me about X' for unfamiliar domains

A: Strategy: (1) Acknowledge knowledge gaps honestly ('I haven't worked extensively with ECC hardware, but I've studied the mathematical foundations'), (2) Show learning initiative ('I've researched side-channel attacks and understand the basic concepts'), (3) Connect to familiar concepts ('My AES verification experience with constant-time checking is similar approach to ECC'), (4) Ask clarifying questions ('Are you asking about ECC verification methodology or specific implementation challenges?'), (5) Offer what you know ('While I haven't verified cryptographic protocols in hardware, I've done protocol validation in firmware and understand the verification challenges.'). Honesty + demonstrated learning path = credibility.

Q: Handling technical questions you can't fully answer

A: Don't: make up answers, pretend to know, over-claim. Do: (1) Acknowledge—'That's a great question. I haven't worked directly with formal verification for information-flow properties, though I understand the concept.' (2) Show understanding—demonstrate what you do know, (3) Contextualize—'In my SERV verification, I used model checking properties for simple state machines. Information-flow formal verification is more advanced; I'm eager to learn.' (4) Ask clarifying—'Could you elaborate on what you mean by information-flow formalization?' (5) Offer related experience—'The closest I've done is prove timing properties with SVA assertions.' This approach shows confidence, honesty, and learning orientation.

Q: Questions to ask the interviewer

A: Technical depth: 'What formal verification approaches does Qualcomm use for security IP? Is there established methodology or are you developing approaches?' Team culture: 'How much time do verification engineers spend hands-on coding vs. meetings?' Tools: 'What simulation tools and formal verification platforms are primary?' Growth: 'What opportunities exist for verification engineers to grow into architectural roles?' Security focus: 'What are the key security challenges your team is tackling in the next 12 months?' Cork office: 'What's the trajectory for the Ireland office—growth plans?' Avoid: salary, benefits, questions easily answered on website, 'I have nothing to ask.'

60-Minute Interview Structure & Tips

- Interviewer (Colm O'Keeffe) indicated 'high-level' discussion—start with clear, concise answers without jargon overload. Be ready to dive deeper if asked specific questions.
- Likely structure: 5-10 min intro/yourself, 20-30 min SERV/AES/projects, 10-15 min verification methodology, 10-15 min security-specific topics, 5-10 min your questions for them.
- Emphasize verification rigor and systematic approach—critical for security hardware where 'good enough' doesn't exist.
- Be honest about security gaps but show learning trajectory. 'I haven't specialized in side-channel analysis YET, but I've studied fundamentals and am committed to deepening expertise.'
- Practice answers to feel natural, not scripted. Let SITUATION/TASK/ACTION/RESULT structure guide you internally.
- Have 3-5 thoughtful questions prepared—shows genuine interest and helps assess fit.
- If asked unexpected questions: Take a moment, ask clarifying questions, structure your answer, show your thinking process. Interviewers want to see how you approach unfamiliar problems.
- Close with enthusiasm: 'I'm genuinely excited about this opportunity. Security hardware verification at Qualcomm represents the kind of impactful work I want to dedicate my career to.'